Vertical AI Agent — Project Roadmap

A custom Q&A chatbot trained on your own content, deployed on your website

What We're Building

A chatbot embedded on your website that only answers using your own business content (docs, FAQs, product info, policies, etc.) instead of generic knowledge. It works by storing your content in a searchable vector database, retrieving the most relevant pieces for each question, and having an AI model write a natural answer grounded in that retrieved content. This approach is called **RAG (Retrieval-Augmented Generation)**.

Tech Stack

Component	Tool	Purpose
Vector database	Chroma Cloud	Stores and searches your content as embeddings
LLM + embeddings	Google Gemini API (Flash)	Generates answers and turns text into vectors
App framework	Next.js (App Router)	Website, chat UI, and backend API routes
Hosting / deployment	Vercel	Makes the app live on the internet
Editor	VS Code	Where all the code is written (Windows or Linux)

How It Works, In Order

1. You feed your content (docs, FAQs, product info) into an ingestion script.
2. The script chunks the content and converts each chunk into a vector using Gemini's embedding model.
3. Those vectors are stored in a Chroma Cloud collection — a searchable "filing cabinet."
4. A website visitor asks a question in the chat widget.
5. The app embeds the question, searches Chroma Cloud for the most relevant chunks.
6. Those chunks + the question are sent to Gemini, which writes a grounded answer.
7. The answer streams back to the visitor in the chat widget on your website.

Phased Roadmap

Phase 1 — Accounts & Keys — Day 1

- Chroma Cloud account + API key (already set up)
- Gemini API key from Google AI Studio (free tier)
- Vercel account, connected to a GitHub repo

Phase 2 — Project Scaffolding — Day 1–2

- Create project folder, open in VS Code
- Create Next.js app (App Router) inside a /web folder
- Set up Python virtual environment inside an /ingestion folder
- Install Chroma client SDK and Gemini SDK in both places as needed
- Store all keys in .env files (never commit these to Git)

Phase 3 — Data Ingestion Pipeline

Day 2–3

- Load your source content (docs, PDFs, scraped pages, etc.)
- Chunk content into small pieces (~300–800 tokens, slight overlap)
- Embed each chunk using Gemini's embedding model
- Upsert vectors into your Chroma Cloud collection
- Test on a small subset first before running the full ingest

Phase 4 — Agent / API Layer

Day 3–5

- Build a /api/chat route in Next.js
- Embed incoming user question, query Chroma Cloud for relevant chunks
- Build a prompt: vertical system prompt + retrieved context + question
- Call Gemini Flash to generate a grounded, streamed answer
- Add guardrails so it only answers from your content

Phase 5 — Frontend Chat Widget

Day 4–6

- Build chat UI using the Vercel AI SDK's useChat hook
- Style it to match your website
- Package as an embeddable widget if hosted separately from your main site

Phase 6 — Deploy

Day 6

- Push code to GitHub
- Import repo into Vercel, add environment variables in the dashboard
- Deploy and test the live URL end-to-end

Phase 7 — Embed on Website

Day 6–7

- Add the chat widget to your live website via script/iframe, or link directly if the Next.js app is your site

Phase 8 — Monitor & Iterate

Ongoing

- Watch Chroma Cloud credit balance ($5 free credit)
- Watch Gemini free-tier daily request quota
- Review real user questions, refine chunking and prompts over time

Budget Notes

- Chroma Cloud: $5 free credit, then usage-based billing (writes are the most expensive meter — avoid re-running full ingestion repeatedly).
- Gemini API: Flash and Flash-Lite models are free with daily request limits; Pro models are paid-only.
- Vercel: free hobby tier is sufficient for this project's traffic level.
- Total expected cost to build and run a small version of this project: effectively $0, as long as usage stays within free tiers/credits.

Note: pricing and free-tier terms for Chroma Cloud and Gemini API can change — verify current limits before scaling usage in production.